Estruturas de Controle

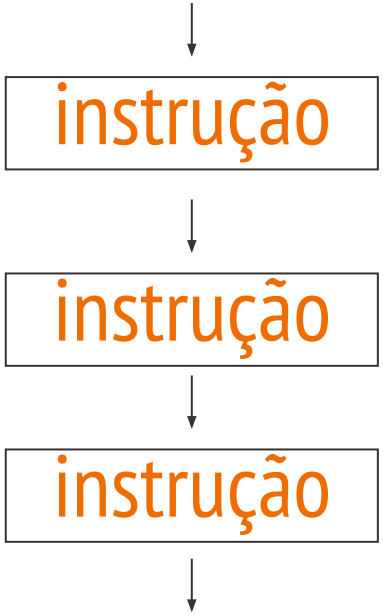
Estrutura Sequencial, Condicional e Repetição

Sumário

- ESTRUTURA SEQUENCIAL
- ESTRUTURA CONDICIONAL
- ESTRUTURA DE REPETIÇÃO

Estrutura Sequencial

Fluxograma



Linguagem Python

```
x,y=2,3

y,x=x,y

print(x,y)
```

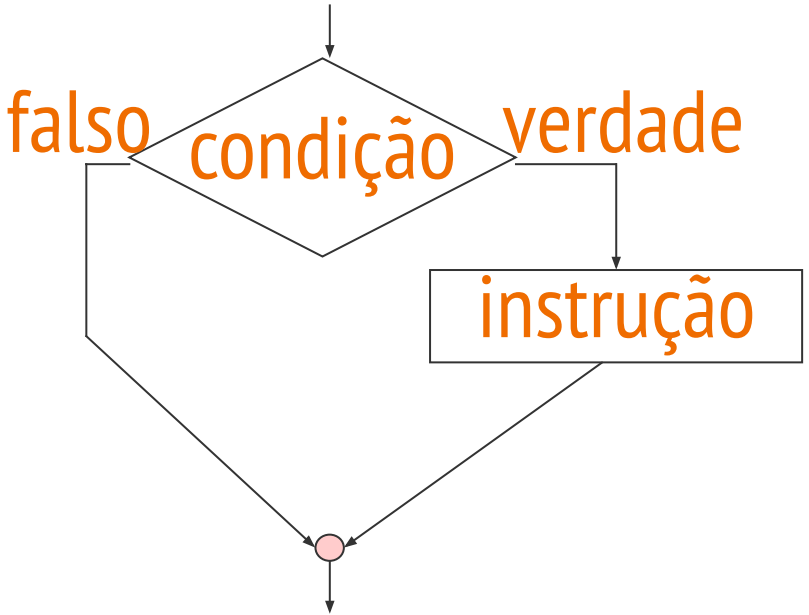
Estruturas Condicionais

- Estrutura Condicional Simples

- Estrutura Condicional Composta

Estruturas Condicionais Simples

Fluxograma



Linguagem Python

```
if (condição):
    <instrução>
```

Estruturas Condicionais Simples

Linguagem Python

```
if (condição):
    <instrução>
```

a condição deve ser uma expressão lógica

A instrução só será executado se a condição for verdadeira

Estruturas Condicionais Simples

Linguagem Python

```
if (condição):
    <instrução>
```

se mais de uma instrução deve ser executada quando a condição for verdadeira, essas instruções devem ser transformadas em instruções compostas.

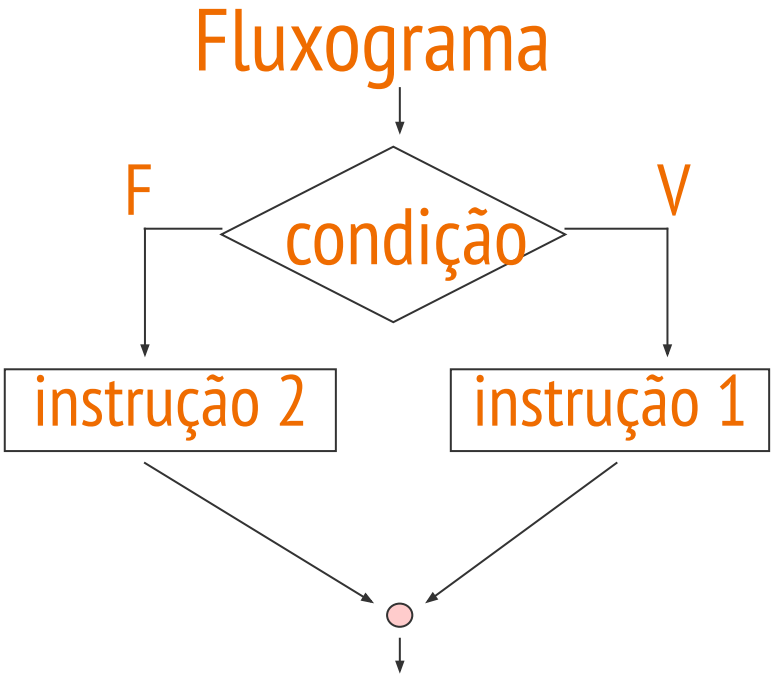
Estruturas Condicionais Simples

Linguagem Python

```
if (condição):
 <instrução>
 <instrução>
 <instrução>
```

As instruções compostas a serem executadas dentro de uma condição devem estar indentadas

Estruturas Condicionais Compostas



Python

```
if (condição):
    <instrução 1>
else:
    <instrução 2>
```

Estruturas Condicionais Compostas

```
if (condição):
        <instrução>
        ...
      <instrução>
else:
        <instrução>
        ...
      <instrução>
```

```
if (condição):
        <instrução>
        ...
      <instrução>
elif(condição):
      <instrução>
      if(cond):
          <instrução>
      else:
          <instrução>
else:
        <instrução>
        ...
      <instrução>
```

Estruturas Condicionais Compostas

a condição deve ser uma expressão lógica

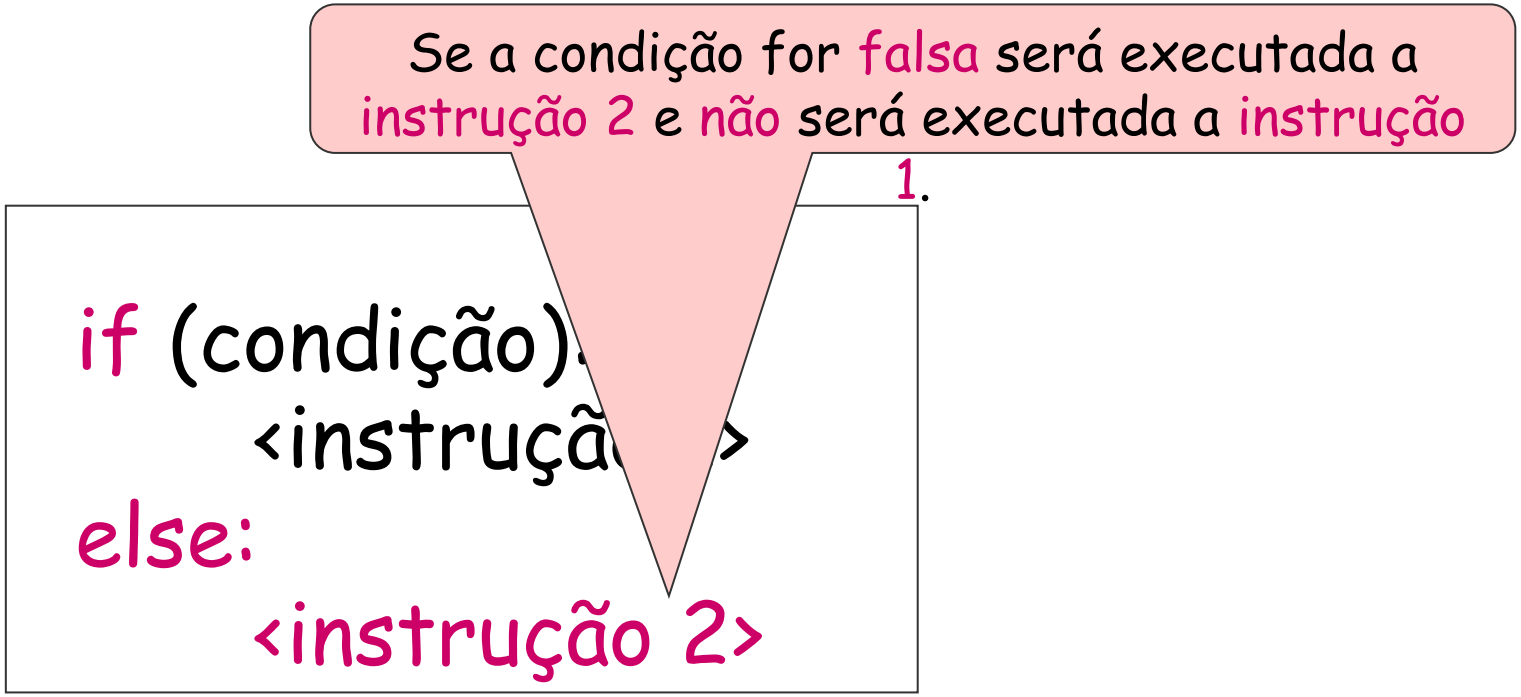
```
if (condição):
        <instrução 1>
else:
        <instrução 2>
```

Estruturas Condicionais Compostas

Se a condição for verdadeira será executada a instrução 1 e não será executada a instrução 2.

```
if (condição):
        <instrução 1>
else:
        <instrução 2>
```

Estruturas Condicionais Compostas



Estruturas Condicionais Compostas

Operador Ternário

<expr1> if <conditional_expr> else <expr2>

```
if (condição):
    <instrução 1>
else:
    <instrução 2>
```

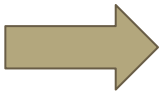


```
<instrução 1> if (condição) else <instrução 2>
```

Estruturas Condicionais Compostas

Operador Ternário

```
nota = 10
if (nota > 5.0):
  print('Aprovado')
else:
  print('Reprovado')
```



```
nota = 10
print('Aprovado') if (nota > 5.0) else print('Reprovado')
nota = 3
msg = 'Aprovado' if (nota > 5.0) else 'Reprovado'
print(msg)
```

Estruturas Condicionais Compostas

pass

```
if (condição):
    pass

<instrução>
```

- If deve ser seguido por alguma instrução.
- A ausência de instrução leva a um erro.
- Utiliza-se pass, se há alguma razão para não incluir uma instrução.

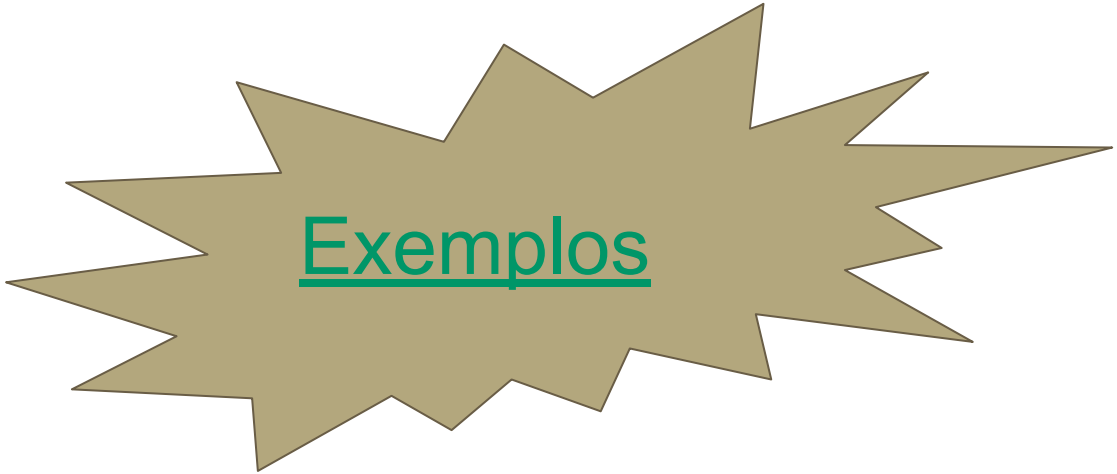
Estruturas Condicionais Compostas

pass

```
#Uso do pass
for number in range(1, 100):
    if number % 13 != 0:
        # Não é múltiplo de 13
      pass
    else:
        print(f'{number} é um múltiplo de 13')
```

```
13 é um múltiplo de 13
26 é um múltiplo de 13
39 é um múltiplo de 13
52 é um múltiplo de 13
65 é um múltiplo de 13
78 é um múltiplo de 13
91 é um múltiplo de 13
```

Estruturas Condicionais Compostas



Estruturas de Repetição

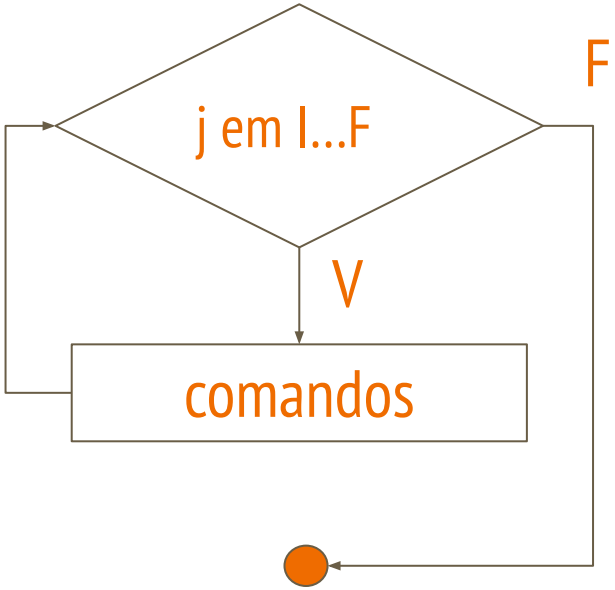
- Uma estrutura de repetição é utilizada quando um comando ou um bloco de comandos deve ser repetido.

- A quantidade de repetições pode ser fixa ou pode depender de uma determinada condição.

Estruturas de Repetição

Repetição Contada

Fluxograma



Linguagem Python

```
for j in range(I,F):
    <comando>
    ...
    <comando>
```

Estruturas de Repetição

```
for <variável-contador> in range(<valores_possíveis>):
        instrução
```

```
for <variável-contador> in range(<valores_possíveis>):
        instrução_1
        instrução_2
        ...
        instrução_n
```

Estruturas de Repetição

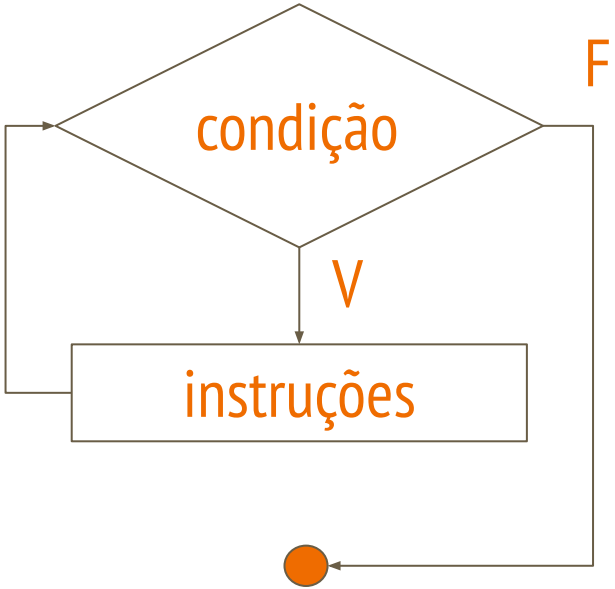
range(início,fim,passo)

- início: primeiro elemento do conjunto de valores
  - default: início = 0
- fim: último elemento do conjunto de valores
- passo: tamanho do incremento ou decremento
  - default: passo = 1
- A repetição persiste até contador = fim -1

Estruturas de Repetição

Repetição com teste no início

Fluxograma



Linguagem Python

```
while(<condição>) :
    <instrução>
    ...
    <instrução>
```

Estruturas de Repetição

"while" pode substituir o comando "for"

- ➔ Inicialização da variável de teste - início
- ➔ while(teste-fim):
  - ◆ incremento da variável de teste - passo

Estruturas de Repetição

"while" pode substituir o comando "for"

for i in range(a,b,passo): <instrução>	i = a while(i!=b): <instrução> i+=passo #a e b números

Estruturas de Repetição

“while” pode substituir o comando “for”

```
for i in range(-10,11,5):
    print("",i,end='')
```

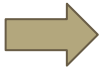
```
i=-10
while(i<11):
  print(f' {i}',end='')
  i+=5
```

```
 -10 -5 0 5 10
```

Estruturas de Repetição

Range não aceita parâmetros float

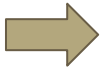
```
for i in range(0,1,0.1):
    print("",i,end='')
print('\n******\n')
```



```
----> 3 for i in range(0,1,0.1):
      4     print("",i,end='')
      5 print('\n******\n')

TypeError: 'float' object cannot be interpreted as an integer
```

```
1  import numpy as np
2  print(f'[0,1]:',end='')
3  for i in np.arange(0, 1, 0.1):
4      print(f'{i:.1f}', end=', ')
5  print(f'\n\n[5.5,15.5]:',end='')
6  for i in np.arange(5.5, 15.5, 1.5):
7      print(f'{i:.2f}', end=' ')
8  print(f'\n\n[15.5,1.5]:',end='')
9  for i in reversed(np.arange(5.5, 15.5, 1.5)):
10     print(f'{i:.2f}', end=' ')
```



```
[0,1]:0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9,

[5.5,15.5]:5.50 7.00 8.50 10.00 11.50 13.00 14.50

[15.5,1.5]:14.50 13.00 11.50 10.00 8.50 7.00 5.50
```

Estruturas de Repetição

numpy.linspace() criar sequências numéricas igualmente espaçadas dentro de um intervalo especificado.

- Útil na geração de eixos de gráfico, discretização de domínios contínuos, simulações numéricas, entre outros.

```
numpy.linspace(start, stop, num=50, *, endpoint=True, retstep=False, dtype=None, axis=0)
```

Parâmetro	Tipo / padrão	O que faz
`start`	escalar	Valor inicial da sequência.
`stop`	escalar	Valor final da sequência.
`num`	int, *50*	Número de pontos gerados.
`endpoint`	bool, *True*	Inclui (`True`) ou não (`False`) o valor `stop` .
`retstep`	bool, *False*	Se `True` , retorna também o espaçamento entre pontos.
`dtype`	tipo NumPy	Tipo dos elementos (por padrão, inferido).
`axis`	int, *0*	Eixo ao longo do qual a sequência é colocada (útil em arrays >1 D).

```python
print(f'[1.5,11.5] com 5:',end=" ")
for i in np.linspace(1.5, 11.5, num=5):
    print(f'{i:.2f}', end=', ')
print('')

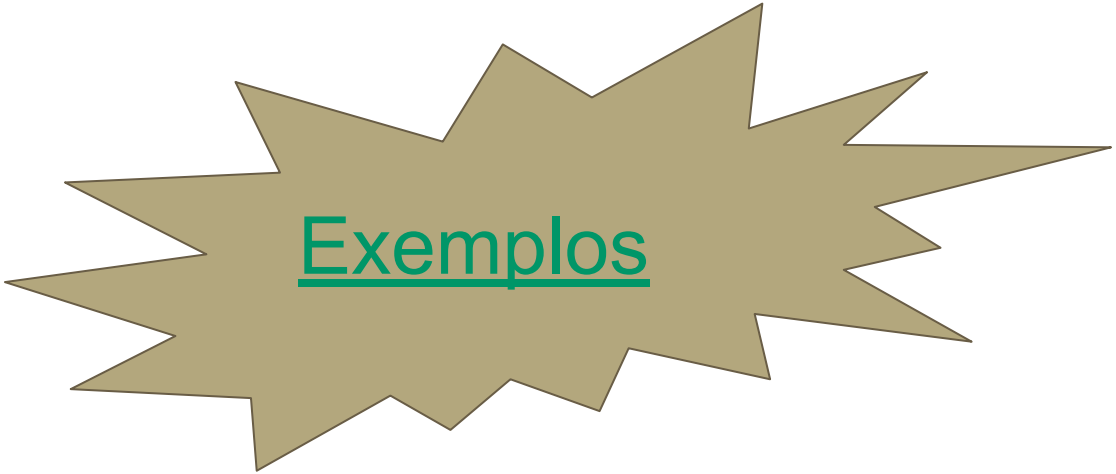
print(f'[1.5,11.5] com 7:',end=" ")
for i in np.linspace(1.5, 11.5, num=7):
    print(f'{i:.2f}', end=', ')
print('')


print(f'[1.5,11.5] com 5 sem incluir último valor:',end=" ")
for i in np.linspace(1.5, 11.5, num=5, endpoint=False):
    print(f'{i:.2f}', end=', ')
print('')

print(f'[1.5,11.5] com 7 sem incluir último valor:',end=" ")
for i in np.linspace(1.5, 11.5, num=7, endpoint=False):
    print(f'{i:.2f}', end=', ')
print('')
```

```
[1.5,11.5] com 5: 1.50, 4.00, 6.50, 9.00, 11.50,
[1.5,11.5] com 7: 1.50, 3.17, 4.83, 6.50, 8.17, 9.83, 11.50,
[1.5,11.5] com 5 sem incluir último valor: 1.50, 3.50, 5.50, 7.50, 9.50,
[1.5,11.5] com 7 sem incluir último valor: 1.50, 2.93, 4.36, 5.79, 7.21, 8.64, 10.07,
```

Estruturas de Repetição



Estruturas de Repetição

Comando break

- Utilizado no corpo de qualquer estrutura de laço.
- Causa a saída imediata do laço, desviando o programa para a próxima instrução após o laço atual.
- Se estiver em laços aninhados, o break afetará somente o laço que o contém e seus laços internos.
- No caso de laços como for e while, o laço é interrompido e os comandos do programa são retomados a partir da primeira linha fora do laço.

Estruturas de Repetição

Comando break

```
1   import math
2   sum=0
3   while(1):
4       x=int(input("x="))
5       if(x<0):
6           break
7       sum += math.sqrt(x)
8   print("sum=",sum)
```

Estruturas de Repetição

Comando continue

- Força a execução da próxima iteração do laço, não executando o código que vem a seguir.
- Esse comando ocorre apenas nos comandos “for” e “while”.

```
1   for valor in range(1,10):
2       if(valor%2):
3           continue
4       print(valor,end=' ')
```

```
2 4 6 8
```

Estruturas de Repetição

Comando continue

- No caso do “while”, a execução é desviada para o teste condicional e depois segue para o corpo do laço.
  - Interrompe a sequência de execuções dentro do laço e verifica a condição.
- No caso do “for”, o desvio é feito para o incremento, seguido pelo teste condicional e corpo do laço.
  - Interrompe a sequência de execuções dentro do laço.
  - Incrementa ou decrementa o contador, verificando a condição.

Estruturas de Repetição

```
1   for valor in range(1,10):
2       if(valor%2):
3           continue
4       print(valor,end=' ')
```

```
2 4 6 8
```

```
1   valor=1
2   while(valor<10):
3       if(valor%2):
4           continue
5       valor+=1
6       print(valor,end=' ')
```



```
1   valor=1
2   while(valor<10):
3       if(valor%2):
4           valor+=1
5           continue
6       valor+=1
7       print(valor,end=' ')
```



```python
layers = [
    [[1, 2, 3], [4, -1, 6], [7, 8, 9]],  # camada 0
    "INVALID",                            # camada 1 (inteira ignorada)
    [[10, 11, None], [12, 13, 14], [15, -5, 16]]  # camada 2
]

total = 0

for L, layer in enumerate(layers):
    # Nível 1: pular a camada inteira se inválida
    if layer == "INVALID":
        print(f"Camada {L}: inválida -> continue (laço das camadas)")
        continue

    for r, row in enumerate(layer):
        # Nível 2: pular a linha inteira se houver None
        if any(x is None for x in row):
            print(f"  Linha {r}: contém None -> continue (laço das linhas)")
            continue

        for c, x in enumerate(row):
            # Nível 3: pular só este elemento se for negativo
            if x < 0:
                print(f"    Coluna {c}: negativo ({x}) -> continue (laço das colunas)")
                continue

            total += x

print("Total acumulado =", total)  # -> 110
```



Estruturas de Repetição

Não confunda pass, continue e break

- pass permite incluir uma condição sem instruções relacionadas, mas que podem ser incluídas no futuro (placeholder)
- break altera o fluxo de execução do laço que termina quando certa condição é satisfeita.
- continue desvia o fluxo de execução para a próxima iteração.